SECTION 1: MVP SCOPE MATRIX & USER WORKFLOWS

Operational Boundaries of the 1st MVP The Phase 1 MVP is strictly scoped to handling intake legality, basic medical documentation, legally compliant crowdfunding, and automated tax-exempt sponsor rewards. Features like predictive animal behavior matching are strictly out-of-scope, as subjective behavior forecasting creates significant liability risks for the platform and the shelter.

Step-by-Step User Journeys

- **Role 1: General Public (Stray Finder / Owner Surrender)**
    - **Step 1:** User accesses the public portal and selects "Surrender" or "Found Stray".
    - **Step 2:** User is presented with a UETA-compliant consumer consent disclosure, requiring explicit agreement to conduct electronic business before proceeding.
    - **Step 3:** User fills out intake details. For owner surrenders, the UI mandates an affirmative action (e.g., typing a name or drawing a signature) to execute a legally binding "Transfer of Ownership Waiver".
    - **Step 4:** The system generates a cryptographic hash of the document's state to link the signature inextricably to the record, creating an immutable PDF snapshot stored in AWS S3.
- **Role 2: County Animal Control Officer (Formal Digital Handoff)**
    - **Step 1:** Officer logs in via an authenticated portal using their badge number and department ID.
    - **Step 2:** Officer executes a municipal transfer to the rescue.
    - **Step 3:** The system verifies the receiving rescue organization's standing, pinging their `ein_number` for IRS compliance and checking local municipal facility zoning permits (e.g., LA County conditional use permits).

- **Role 3: Shelter Administrator / Medical Intake Staff**
    - **Step 1:** Admin reviews the incoming animal and triggers the `Stray_Hold_Calculator` microservice. This engine parses the intake timestamp, facility operating hours, and local GPS/jurisdiction rules to automatically calculate the exact hour the animal legally clears its stray hold.
    - **Step 2:** Staff conducts a brief medical intake exam based on ASPCA guidelines, recording Temperature, Pulse, Respiration (TPR), scanning for a microchip, and noting emergency signs (e.g., vomiting, conjunctivitis).
    - **Step 3:** Admin publishes the animal for crowdfunding. The system cross-references the rescue's `charleston_compliance_json` to enforce geographic blocking, preventing illegal solicitation in states where the non-profit is not registered.

SECTION 2: COMPREHENSIVE DATA SCHEMA & RELATIONAL MODEL

The following relational model is defined using Prisma schema pseudo-code, merging physical tracking with legal/financial constraints.

None

```
// Organizations act as regulatory gatekeepers

model Organization {

  orgId                    String   @id
@default(uuid())

  orgType                  OrgType  //
MUNICIPAL_SHELTER, 501C3_RESCUE, CORPORATE_SPONSOR

  einNumber                String   @unique //
Verified via IRS API

  stateAgLicenseNumber     String?  // Mandatory for
states like IL (PACFA)

  charlestonCompliance     Json?    // Array of state
IDs for solicitation legality

  animals                  Animal[]

}



// Central asset ledger tracking the legal custody
and health of the animal

model Animal {
```

```
  animalId                String   @id
@default(uuid())

  custodianOrgId          String

  organization            Organization
@relation(fields: [custodianOrgId], references:
[orgId])

  breedDesignation        String   @default("Mixed
Breed") // Prevents BSL insurance liability

  intakeTimestamp         DateTime @default(now())

  strayHoldExpiresAt      DateTime?

  statutoryHoldCleared    Boolean  @default(false)
// Strictly controlled by Rules Engine

  medicalStatus           MedState // HEALTHY_HOLD,
ISOLATION, QUARANTINE



  // Relations

  intakeRecord            IntakeRecord?

  medicalLogs             MedicalLog[]

  fundraisingWallet       FundraisingWallet?
```

```
}


// Intake split between authenticated officials and
public citizens

model IntakeRecord {

  intakeId                String   @id
@default(uuid())

  animalId                String   @unique

  animal                  Animal   @relation(fields:
[animalId], references: [animalId])

  intakeType              InType   // PUBLIC_STRAY,
OWNER_SURRENDER, MUNICIPAL_TRANSFER



  // Public Flow

  esignIntentHash         String?

  electronicConsentFlag   Boolean  @default(false)

  documentSnapshotUrl     String?  // AWS S3
Immutable Link
```

```
  // County Flow
  officerBadgeNumber       String?
  departmentId             String?
}

// ASPCA/CART aligned medical check parameters
model MedicalLog {
  logId                    String   @id
@default(uuid())
  animalId                 String
  animal                   Animal   @relation(fields:
[animalId], references: [animalId])
  weightLbs                Float?   //
  tempPulseResp            Json?    // TPR tracking
  microchipScanned         Boolean  @default(false)
//
```

```
  vaccinesAdministered     Json?    //

  dewormerAdministered     Json?    //

  emergencySignsNoted      Json?    // Array of
symptoms (e.g. coughing, patchy hair loss)

}



// Tracks donor intent and automatically restricts
fund mixing

model FundraisingWallet {

  walletId                 String   @id
@default(uuid())

  animalId                 String   @unique

  animal                   Animal   @relation(fields:
[animalId], references: [animalId])

  targetAmount             Decimal

  currentBalance           Decimal  @default(0.00)

  restrictionType          FundType // MEDICAL,
GENERAL_CARE, TRANSPORT
```

```
}


// UBIT compliant vendor perk distribution

model SponsorshipPerk {

  perkId                  String   @id
@default(uuid())

  sponsorOrgId            String

  triggerEvent            Trigger  //
ADOPTION_FINALIZED, DONATION_MADE, REFERRAL_SUCCESS

  uiRenderMode            Render
@default(NEUTRAL_ACKNOWLEDGMENT) // Prevents UBIT
triggers

  rewardCode              String

}
```

SECTION 3: FRONTEND COMPONENT ARCHITECTURE & COMPONENT TREE

Next.js App Router Directory Structure:

- `app/intake/public/page.tsx` (Citizen workflow)
- `app/intake/county/page.tsx` (AC Officer workflow)
- `app/dashboard/page.tsx` (Shelter Admin control center)
- `app/medical/[animalId]/page.tsx` (Veterinary/Intake examination)

Component Tree & Granular Breakdown:

1. **`<ESIGNLiabilityWaiver />`**
    - *Purpose:* Renders the ESIGN/UETA compliant surrender form.
    - *Props:* `animalId: string`, `surrenderType: enum`.
    - *Interactions:* Checkbox for "Consent to Electronic Business". Cannot submit unless affirmative signature action is detected (Framer Motion shake animation on error if bypassed).
    - *Dynamic State:* Generates SHA-256 hash payload onSubmit to feed the `IntakeRecord` database model.
2. **`<StrayHoldTracker />`**
    - *Purpose:* Visualizes the output of the `Stray_Hold_Calculator` microservice for Shelter Admins.
    - *Props:* `intakeTimestamp: string`, `holdExpiresAt: string`, `isCleared: boolean`.
    - *UI:* Renders a locked padlock icon with a countdown timer. The "Adopt" or "Transfer" action buttons remain strictly disabled (greyed out) until `isCleared` is true, preventing accidental illegal transfers.

3. **`<MedicalIntakeChecksheet />`**
    - *Purpose:* Digitizes the ASPCA/CART brief intake exam.
    - *Props:* `animalId: string`, `species: string`.
    - *Interactions:* Toggle switches for "Microchip Scanned", "Vaccines Administered", and "Topspot flea/tick applied". Includes a highly visible "EMERGENCY SIGNS" conditional rendering block; if checkboxes like "Ulcers in mouth/nose" or "Vomiting" are ticked, the UI immediately flags the record for veterinary notification.
4. **`<UBITVendorPerkCard />`**
    - *Purpose:* Delivers corporate sponsor coupons to users after a trigger event (e.g., successful donation).
    - *Props:* `sponsorLogo: string`, `sponsorName: string`, `rewardCode: string`.
    - *Compliance Constraints:* Structurally decoupled from all marketing systems. The component forces `ui_render_mode="NEUTRAL_ACKNOWLEDGMENT"`. It explicitly disables any rendering of qualitative language (e.g., "best pet food") or price comparison data (e.g., "20% off") to strictly prevent classifying the transaction as taxable advertising income under IRS IRC Section 513(i).

SECTION 4: STEP-BY-STEP DEVELOPER IMPLEMENTATION ROADMAP

This roadmap is designed for an autonomous coding agent to execute sequentially, minimizing compilation errors.

- **Step 1: Scaffolding & Database Models**
    - *Task:* Initialize Prisma schema based on the provided relational blueprint.

    - *Constraint:* Ensure all Primary Keys are UUIDs and enumerations (OrgType, MedState) are strictly defined. Run `prisma generate` and apply the initial migration.
- **Step 2: Microservice - The Stray Hold Rules Engine**
    - *Task:* Build the isolated `Stray_Hold_Calculator` utility function.
    - *Constraint:* It must accept an intake timestamp and a jurisdiction code, returning a computed expiration DateTime and a boolean flag.
- **Step 3: Shared UI Layouts & Compliance Wrappers**
    - *Task:* Construct the Next.js App Router layout shells (`app/intake/layout.tsx`, `app/dashboard/layout.tsx`).
    - *Constraint:* Build the `<UBITVendorPerkCard />` component first, ensuring it only accepts props for a logo and a neutral text string to guarantee tax compliance.
- **Step 4: Public & County Intake Forms (ESIGN Integration)**
    - *Task:* Develop `app/intake/public/page.tsx` and `app/intake/county/page.tsx`.
    - *Constraint:* Implement the `<ESIGNLiabilityWaiver />` component with active state validation to guarantee intent-to-sign and digital consent.
- **Step 5: Medical Logging Views**
    - *Task:* Implement `app/medical/[animalId]/page.tsx` using the `<MedicalIntakeChecksheet />`.
    - *Constraint:* Wire the frontend form to update the `MedicalLog` Prisma model, ensuring TPR and emergency sign data are accurately captured.
- **Step 6: Fundraising Wallet Logic**
    - *Task:* Build the backend mutations that update `FundraisingWallet.currentBalance`.

- *Constraint:* Implement the automated restriction logic; once `currentBalance` hits `targetAmount`, the frontend must switch the `restrictionType` flag to prevent mixing of legally restricted donor assets.